

QMedViT: The First Steps Towards A Quantum Computing Adaptation of MedViT — Hybrid CNN-Vision Transformer Architecture for Medical Image Classification

Alexandra-Paraschieva Chiriță

*Computer Science and Engineering Department, University Politehnica of Bucharest, Bucharest, Romania.

Corresponding author(s). E-mail(s):
alexandra.chirita98@stud.acs.upb.ro;

Abstract

State-of-the-art machine learning architectures achieve high performance on medical datasets. Still, certain noisy and imbalanced datasets present a challenge for accurate disease classification from images. One of the premises of quantum advantage in machine learning is better representation of the feature space, which could also improve image classification. The present work analyses a robust CNN-Transformer architecture and attempts to replace classical blocks with quantum methods. While the quantum implementation slightly underperforms the classical models, it shows the potential to improve on GPU training and optimization. Implementation available at <https://github.com/alexandrachirita98/MedViT-Quantum>.

1 Introduction

Quantum Computing presents potential in several application domains, such as molecular simulation, quantum sensing, cryptography, financial modeling, and logistics optimization [1]. Recent surveys [2, 3] highlight the potential advantages of Quantum Machine Learning (QML) algorithms: enhanced feature mapping due to the use of quantum states, and comparable or superior accuracy of QML algorithms, with fewer parameters compared to classical algorithms (i.e., autoencoders). QML still



faces numerous limitations, such as hardware capabilities, limited qubit counts, noisy quantum systems, and a lack of mature accuracy results [1].

Recent results in the RetinaMNIST and DermaMNIST datasets [4–8] indicate a difficulty for state-of-the-art architectures to classify certain diseases with high accuracy. These results inspired the approach of replacing elements from an existent robust image classification architecture with quantum-based elements.

In this report, we present the adaptation of 2 blocks of the robust medical image classification architecture MedViT [4]: the convolutional layer and attention mechanism. We decided to work on MedViT, as it brings improved results on the MedMNISTv2 datasets. Our work will evaluate the performance of the quantum variants derived from MedViT using only the RetinaMNIST dataset.

2 MedViT

MedViT [4] is a CNN-Transformer hybrid model, which makes use of the advantages and mitigates the disadvantages of the two architectures. While CNNs bring lower computational complexity and can handle vast volumes of data, their locality bias makes it difficult to learn long-range dependencies in images. Transformer architectures model long-range dependencies in images due to the self-attention mechanism. However, transformers are computationally complex, which is infeasible for high-resolution images.

The study architecture MedViT comprises 3 major architectural blocks. First, we have the Efficient Convolution Block (ECB), which is a hybrid component that combines both the representational power of the Transformer block with the efficiency and inductive biases of the residual block [4, §3.2].

The Local Transformer Block (LTB) [4, §3.3] specialises in capturing global information. The block uses a convolutional Multi-Head Self-Attention (MHCA) for effective local representation learning. MHCA consists of concatenated single-head convolutional attention, references in Equation 1. In the present work, we study the effect of quantum methods on the attention mechanism in Section 4.

$$\begin{aligned} \text{MHCA}(x) &= \text{Concat}(\text{CA}_1(x_1), \text{CA}_2(x_2), \dots, \text{CA}_h(x_h)) W^O \\ \text{CA}(X) &= (W \cdot T_{\{i,j\}}), \quad \text{where } T_{\{i,j\}} \in X \end{aligned} \tag{1}$$

While not subject to our study, the 3rd major block in MedViT is the Transformer Augmentation Block (TAB) [4, §3.4] — a data-augmentation mechanism applied in feature space during training. Lastly, in our work, we also perform a quantum transformation of the initial patch embedding stem in Section 3, which plays a role in the image encoding from [4, Figure 2].

In 2025, the authors enhanced the architecture in MedViTV2 [5], which integrated Kolmogorov-Arnold Network (KAN) layers into the Transformer architecture.



3 Convolutional Block

The present work attempts to first change the 2D convolution in the convolutional block `ConvBNReLU`. The convolutional block is composed of 3 components. First, the block applies a 2D convolutional layer, which extracts local features from the input feature map [9]. Next, the block applies batch normalization (BN) [10], which recenters and rescales the convolutional layer's per-channel outputs to stabilize the pre-activation distributions fed into the subsequent Rectified Linear Unit. The MedViT [4] implementation uses the default PyTorch classes `Conv2d` [11] for the convolutional layer, `BatchNorm2d` [12] for batch normalization, and `ReLU` [13] for the Rectified Linear Unit (ReLU). According to [14], ReLU is the activation function, which maps all negative values to 0

$$\text{ReLU}(x) = \max(0, x), \quad (2)$$

In [15, §2.2.2], the 2nd postulate of quantum mechanics states that:

The evolution of a closed quantum system is described by a unitary transformation.

Batch normalization and Rectified Linear Unit don't preserve the norm, hence they cannot be expressed using **unitary** quantum operations:

$$\begin{aligned} \text{BN} : \quad \|x\|_2 = 2 &\xrightarrow{\text{E}[x]=1, \text{Var}[x]=1}} \|y\|_2 = \sqrt{2}, \\ \text{ReLU} : \quad \|[3, -4]\|_2 = 5 &\longrightarrow \|[3, 0]\|_2 = 3. \end{aligned}$$

where $\text{E}[x]$ and $\text{Var}[x]$ denote the per-channel mean and variance of x computed across the batch and spatial dimensions.

ReLU violates 2 more fundamental tests for quantum operations. According to the same 2nd postulate of quantum mechanics from [15, §2.2.2], the evolution of a closed system could be described by a unitary matrix U , which performs linear operations. ReLU performs non-linear operations. A linear operation f satisfies:

$$f(\alpha x + \beta y) = \alpha f(x) + \beta f(y). \quad (3)$$

for any scalars α, β and vectors x, y . ReLU doesn't satisfy this property. For instance, taking $x = 1$, $y = -2$, and $\alpha = \beta = 1$, we have:

$$\text{ReLU}(x + y) = 0 \neq 1 = \text{ReLU}(x) + \text{ReLU}(y).$$

ReLU is therefore non-linear and cannot be implemented as a unitary quantum operation [15, §2.2.2]. Another feature of quantum gates is that they are invertible ($U^\dagger U = I$) [15, §2.2.2]. In the PyTorch documentation [16] and the cornerstone references [14, 17, 18], Given Equation 2, ReLU is non-injective, hence non-bijective and non-invertible [19].



3.1 Untrainable Quantvolutional Layer

According to [20], the random nonlinear features produced by quanvolutional circuits could improve the accuracy of the trained models. The work improved the accuracy and faster decrease of log-loss with each iteration, compared to a classical CNN.

The classical convolution [9] as implemented by [11, 16], `Conv2d.forward(x)` first extracts $k \times k$ patches from the input via a sliding window, then computes the scalar product $W_{patch} + b$ on each position and channel output, finally returning the feature map. Based on [20], our implementation of the quanvolutional layer `Quanv2d`, which replaces the `Conv2d` classical convolution, implements a modified `forward` method and an additional `_build_quanv_circuit` function, which builds the random quantum circuit.

3.1.1 Random Quantum Circuit

In the first step of the implementation, we built a method that generated fixed random circuits for each output channel for the quanvolution. The present variant uses the same 3-by-3 quanvolutional filters and simulates on a 9 qubit circuit [20, §3.4].

In order to generate the circuit, the algorithm treats every qubit as a node in an unoriented graph and assigns a connection probability that a 2-qubit gate will be applied between 2 qubits. We use the Erdős–Rényi model [21], which determines by an independent probability between nodes if the algorithm picks an edge. In the circuit generation process, we use 1 and 2-qubit gates, mapped by the PennyLane primitives: `qml.CNOT`, `qml.SWAP`, `qml.SISWAP`, `qml.CRot`, `qml.RX`, `qml.RY`, `qml.RZ`, `qml.Rot`, `qml.S`, `qml.T`, `qml.Hadamard` [22]. The number of 1-qubit gates is generated randomly via a uniform distribution in $[0, 2 \cdot n_{\text{qubits}}]$. The rotation angles range between $[0, 2\pi]$. At the end, the function that builds the circuit `_build_quanv_circuit` shuffles the generated gates [20, §3.4]. Reference Figure 1 for an example of a random quantum circuit created by the algorithm from [20].

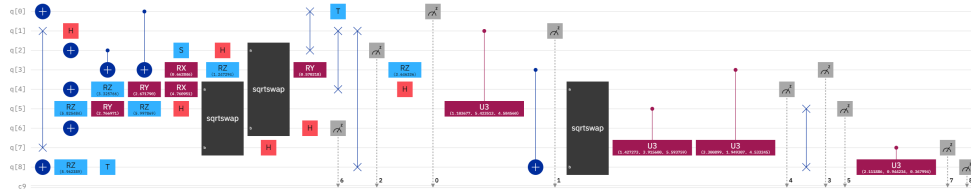


Fig. 1: Random Quantum Circuit generated by the algorithm implemented in [20].

3.1.2 Encoding & Decoding

After the circuit generation, the algorithm from [20, §3.5] performs the encoding and decoding operations. Encoding maps classical data into a quantum state via a feature map [23, 24]. We adopt the basis encoding [20, §3.5], in which each binarized pixel sets its corresponding qubit to a computational basis state $|0\rangle$ or $|1\rangle$ [25]. The binary

encoding scheme from [20, §3.5] applies a Pauli X gate on the pixel over a given threshold. The decoding transforms the classical data into a quantum state [15]. For each qubit, we measure the expectation value of the state vector. Our implementation deviates from [20, §3.5] by using mean of $\langle Z \rangle$ from [22].

3.1.3 Forward Function

The forward function in `Quanv2d` keeps the classical convolution's scaffolding. In the classical `Conv2d` from MedViT [4], the scalar product uses the real pixels directly, with no encoding. As per the proposal from [20], we apply bias encoding over the patches, making the input of the quantum circuit finite and determinable (see Figure 2).

```
# Conv2d: no binarization.
# Float values go straight in:
out = sum(W * patch) + b
# patch holds real values
```

(a) Classical convolution.

```
# Quanv2d: threshold encoding.
bits = (patches.detach()
        > self.threshold).to(torch.
        uint8)
```

```
def _apply_encoding(bits):
    for q in range(n_qubits):
        if int(bits[q]) == 1:
            qml.PauliX(wires=q)
```

(b) Quantum threshold encoding.

Fig. 2: Classical convolution uses the pixel values directly into the linear product, while the quanvolutional layer first converts each pixel into a binary value, based on a threshold, then maps the bit to a qubit via `PauliX`.

Instead of the linear scalar product $Wpatch + b$ from the original MedViT implementation, the quantum variant simulates the operation on the fixed random quantum circuit, as shown in Figure 3.

```

# Linear, trainable, continuous
input:
out[:, oc] = sum_ic(
    (W[oc, ic] * patch_ic).sum()
) + b[oc]
# No circuit

(a) Classical convolution (per output chan-
nel).

# Run the quantum circuit per unique
bitstring:
scal = np.empty(unique_keys.shape[0],
    dtype=np.float32)
for u in range(unique_bits.shape[0]):
    z = qnode(unique_bits[u]) # quantum
    sim
    z = np.asarray(z).reshape(-1)
    p1 = 0.5 * (1.0 - z) # <Z> -> P
    (/1>)
    scal[u] = float(p1.mean()) # mean
    over qubits

scalars = scal[inverse] # scatter back

(b) Quanvolution: per-bitstring circuit evalua-
tion.
    
```

Fig. 3: Per position operator. The classical convolution computes a linear product, compares to the quanvolutional layer, which evaluates a fixed random quantum circuit.

3.2 Quantvolutional Layer Optimization

The base implementation from [20] poses several challenges in the MedViT integration. Let N_{img} be the number of training images, B the batch size, C_{out} and C_{in} the output/input channel counts of the quanvolutional layer, $U_b \leq \min(B \cdot L, 2^{n_q})$ the number of *distinct* bitstrings in batch b (with L patches per image and $n_q = k^2$ qubits), and τ_{sim} the wall-clock time of a single circuit simulation `qnode(bits)`.

The number of circuit evaluations per epoch is

$$Q_{\text{epoch}} = \underbrace{\left\lceil \frac{N_{\text{img}}}{B} \right\rceil}_{\# \text{ batches}} \times \underbrace{(C_{\text{out}} \cdot C_{\text{in}})}_{\text{circuits / forward}} \times \underbrace{\overline{U}}_{\text{unique bitstrings / batch}}, \quad (4)$$

where $\overline{U}$ is the per-batch average of U_b . The epoch wall-clock time is then approximately

$$T_{\text{epoch}} \approx Q_{\text{epoch}} \cdot \tau_{\text{sim}} = \left\lceil \frac{N_{\text{img}}}{B} \right\rceil \cdot C_{\text{out}} \cdot C_{\text{in}} \cdot \overline{U} \cdot \tau_{\text{sim}}. \quad (5)$$

Before calculating the estimated time per epoch, we outline the number of quantum circuits for the small version of the model. The full calculation per layer is available in Table 1. Based on the measures time for a single quantum circuit evaluation $\tau_{\text{sim}} \approx 3.93$ ms (9 qubits, `depth=2`, 52 gates, `default.qubit`), the estimation for a single epoch exceeds the objective of the present work; to implement quantum variants with training time comparable to the one of the MedViT original implementation [4] (less than 10 minutes per epoch on small model). The unoptimized implementation of [20] is infeasible, according to the Table 2.



Table 1: Number of circuits and lookup-table evaluations per stem layer ($k = 3$, $n_q = 9$, $2^{n_q} = 512$). Each circuit is tabulated over all 512 basis states; the build is performed once.

Layer	$C_{in} \rightarrow C_{out}$	$N_{circ} = C_{out} \cdot C_{in}$	$N_{eval} = N_{circ} \cdot 512$
1 ($3 \rightarrow 64$)		192	98,304
2 ($64 \rightarrow 32$)		2048	1,048,576
3 ($32 \rightarrow 64$)		2048	1,048,576
4 ($64 \rightarrow 64$)		4096	2,097,152
Σ		8384	4,292,608

Table 2: Single-epoch time estimate for the unoptimized quanvolutional base variant (RetinaMNIST, 1080 train images, batch size 10, $\tau_{sim} \approx 3.93$ ms per 9-qubit circuit evaluation).

Step	Calculation	Result
Evals / batch	8384×512	4 292 608
Time / batch	$4\,292\,608 \times 3.93$ ms	16 870 s $\approx$ 4.69 h
Batches / epoch	1080/10	108
Evals / epoch	$4\,292\,608 \times 108$	463 601 664
Time / epoch	$463.6 \text{ M} \times 3.93$ ms	1.82×10^6 s $\approx$ 506 h $\approx$ 21.1 days

3.2.1 Lookup Table of Quantum Circuit

The quanvolutional layer in [20] is static and untrainable. The quantum circuits remain the same for each training step, which enables us to implement a caching method for the circuits via a lookup table, as per the suggestion from [20]. The lookup table is built once before forwards at model initialization, and uses a versioning caching system based on the number of qubits that the model uses for the quanvolutional layer. The number of distinct random circuits sampled is

$$N = C_{out} \cdot C_{in} \cdot 2^{k^2} \quad (6)$$

$C_{out}C_{in}$ being the number of circuits per layer and 2^k the size of the input space for k qubits. Based on the calculation from 1, the precomputation sums ≈ 4.29 million circuit evaluations.

With a parallelized build across 18 worker processes, circuit precomputation completes in the order of minutes, as demonstrated in Equation 7.

$$T_{build} \approx \frac{N_{circ} \cdot \tau_{batch}}{P} = \frac{8384 \times 318 \text{ ms}}{18} \approx 148 \text{ s} \approx 2.5 \text{ min} \quad (7)$$

The present optimization works with already delivered circuits from a remote repository and uses lazy circuit memorization per batch¹. In case the number of qubits exceeds 16, the model falls back onto the base variant to fit memory limitations.

¹For memoization caches see the Python `functools.lru_cache` documentation, https://docs.python.org/3/library/functools.html#functools.lru_cache.

3.2.2 GPU Optimzation and Serialization

Our implementation optimizes the forward function `_forward_lut`. It uses the lookup table to precompute every possible input bit combination from a given patch. For 4 qubits (a 2×2 patch), there are only $2^4 = 16$ possible inputs, as depicted in Table 3.

Table 3: Example lookup table (LUT) for a single frozen circuit with $n_q = 4$ qubits.

key	bits	answer (precomputed)
0	0000	0.13
1	0001	0.51
2	0010	0.47
3	0011	0.62
$\vdots$	$\vdots$	$\vdots$
15	1111	0.88

Once the lookup table is precomputed, the forward pass runs entirely in torch on the model's device, as in Figure 4b. Due to precomputation of the quantum circuits, the implementation doesn't need the QNode² fallback on CPU (see Figure 4a) to evaluate the circuits.

The improved quanvolutional layer variant of MedViT optimized the circuit evaluation by encoding every basis state into a broadcast dimension and evaluating QNode only once per unique bitstring, as shown in Figure 5.

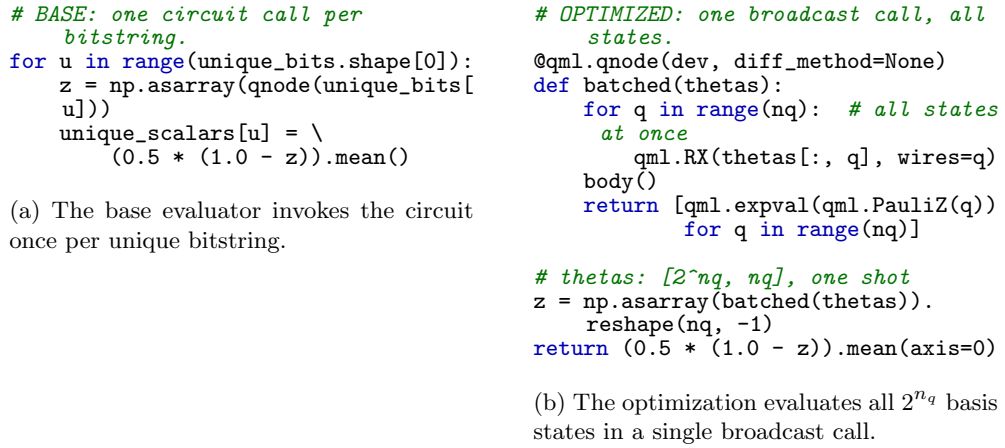


Fig. 5: Lookup-table construction: one circuit call per bitstring (base) versus a single broadcast QNode call over all basis states (optimized).

²Quantum node or QNode is the representation of quantum computation units in PennyLane. For more information https://pennylane.ai/glossary/quantum_node and https://pennylane.ai/demos/tutorial_mcm_introduction


```

# BASE: forward via QNode, CPU round-
trip.
bits = (patches.detach()
        > self.threshold).to(torch.
        uint8)
bits_np = bits.cpu().numpy() # GPU
-> CPU

for oc in range(self.out_channels):
    acc = np.zeros((B, L), dtype=np.
    float32)
    for ic in range(self.in_channels)
    :
        flat = ... # reshape patch
        bits
        # evaluates the PennyLane
        circuit
        scalars = self.
        _eval_qnode_on_unique(
            self._qnodes[oc][ic],
            flat)
        acc += scalars.reshape(B, L)
    out_oc = torch.from_numpy(acc) \
    .to(device=x.device) # CPU
    -> GPU
    out[:, oc] = out_oc.reshape(B,
    H_o, W_o)

# OPTIMIZED: LUT forward, all in
torch.
bits = (patches.detach()
        > self.threshold).to(torch.
        uint8)
# self._lut: precomputed buffer on
GPU
powers = 2 ** torch.arange(
    nq, device=device, dtype=torch.
    long)

# key packing: nq bits -> one integer
keys = ... # (bits * powers).sum
over nq

for ic in range(self.in_channels):
    gathered = self._lut[:, ic, :][:,
    keys[:, ic]]
    acc += gathered.permute(1, 0, 2)
    # gather
    out = acc.view(B, self.out_channels,
    H_o, W_o)
# dtype-only cast; no host<->device
copy
return out.to(dtype=x.dtype)

```

(a) The base quanvolutional layer variant copies for each pass the bitstrings to the CPU, evaluates the frozen circuits in PennyLane, then copies the result back to the GPU.

(b) In the optimized quanvolutional layer, the frozen circuits are precomputed once into a GPU lookup table; with no CPU round-trip and no circuit evaluation.

Fig. 4: The base variant for [20] forces a GPU→CPU→GPU transfer for each PennyLane simulation per patch. The implemented optimization takes advantage of the deterministic circuits' layout and uses the precomputed ones from the lookup table.

3.3 Trainable Quanvolutional Layer

In the present stage of our MedViT's quantum architecture adaptation we attempted to implement a trainable quanvolutional layer based on the quantum convolutional neural network (QCNN) proposal from [26]. The implementation couldn't benefit from the lookup table optimization for the untrainable variant, as it uses continuous angle encoding $RY(\text{angle})$, that change every backward step. For further development of a trainable variant, we should consider further GPU optimization or integrating the full QCNN from [26] instead of the current trainable quanvolutional layer adaptation.

4 Attention Mechanism

The most prominent element in Transformer architectures is the self-attention mechanism [27, 28]. MedViT implements a Local Transformer Block (LTB), specialized in capturing global information and high-frequency signals. LTB uses Efficient-Self

Attention (ESA) and Multi-Head Self-Attention (MHSA) to produce a mix of high-low features [4].

The Scaled Dot-Product Attention (SDPA) is the key component of any Transformer that gives them their global receptive field. The attention operator [4] is defined as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V$$

where $Q, K, V \in \mathbb{R}^{N \times d_k}$ are the input query, key, and value matrices from the linear projection of the tokens, and d_k is the per-head dimension. The softmax turns the scaled similarities into attention weights, used to compute a weighted sum of the value vectors, producing a context-aware representation of the tokens [27].

4.1 Quantum Parameterized Self-Attention

The proposal from [29] replaces the dot-product scoring function in multi-head self-attention — a block present in MedViT's [4] architecture — with quantum parametrized attention (QPA)³, as in Figure 6. The quantum model from [29] shows improved performance over the classical model on noisy datasets such as DirtyMNIST ($92.39\% \pm 0.74\%$ accuracy for the quantum model vs. $91.87\% \pm 0.91\%$ for the classical model).

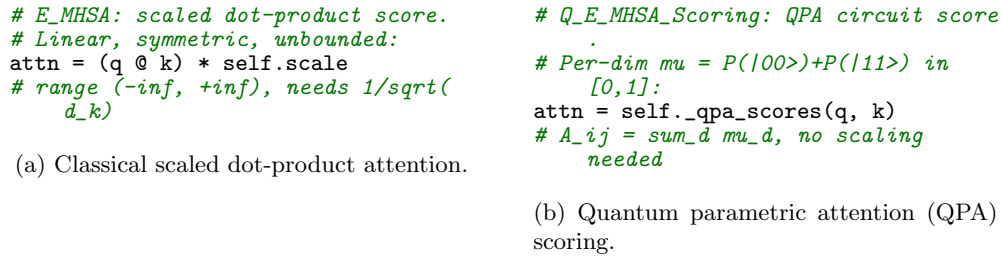


Fig. 6: Classical attention scores each query and key pair with a scaled dot product. The QPA variant instead feeds each pair through a two-qubit parameterized circuit.

QPA comprises 3 steps: encoding, entangling, and measurement. See the full circuit layout in Figure 7. The circuit encodes query-key qubit pairs with the operation $\text{RY}(\varphi_1) \otimes \text{RX}(\varphi_0)$, φ_0 and φ_1 being the encoding angles. Then the circuit entangles the 2 qubits using 2 *CNOT* gates and an *RY* rotation gate, via the unitary transformation

$$U_{\text{ent}}(q, k) = \text{CNOT}_{1 \rightarrow 0} \cdot (I \otimes \text{RY}(\alpha(q + k))) \cdot \text{CNOT}_{0 \rightarrow 1},$$

³We use the attention implementation from the official repository from [29]: <https://github.com/Comets9224/QPSAN>.

where α computes the entangling angle between the 2 qubits. Lastly, the circuit applies RX gates on both qubits, performs the joint measurement in the computational basis as the sum of probabilities of both qubits measuring in the same basis state:

$$\mu_d = P(|00\rangle) + P(|11\rangle) \quad (8)$$

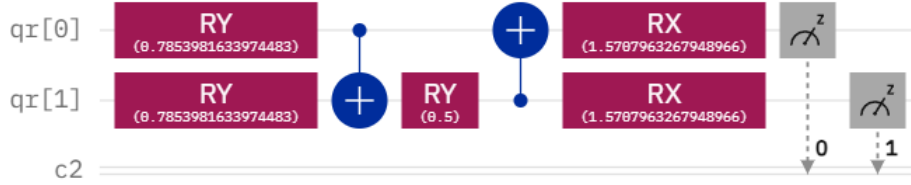


Fig. 7: QPA circuit with encoding angles $\varphi_0 = \varphi_1 = \pi/4$, entangling angle $\theta = 0.5$, and measurement-basis rotations $RX(\pi/2)$ on both qubits; readout in Equation, based on [29].

In the MHSA module, the QPA variant adds 5 trainable parameters per attention layer. Table 4 explains the meaning of each parameter and the initialization values from [29]. The function *qpa_mu* that calculates the per-dimension attention score uses further the trainable parameters. First, the function builds the encoding angles, as per [29, Eqs. 5–6, Lemma 1]. Then, *qpa_mu* encodes the angles into a product state, entangles the 2 states [29, Eqs. 7], mixes the states [29, Eqs. 8], and then performs the measurement, which represents the attention score. Algorithm 1 shows the steps towards the calculation of the attention score. Instead of using the PyTorch⁴ quantum simulator, as in [29], our code optimizes the score calculation with pure torch, on GPU.

⁴https://pennylane.ai/demos/tutorial_state_preparation

Table 4: Trainable parameters of the QPA scoring circuit and their initialization values from QPSAN [29].

Parameter	Description	Paper Init.
θ_s	Initial encoding scale; sets the per-qubit input rotation in the encoding layer.	0.5
γ_d	Difference encoding strength; captures how different the query and key are.	$\mathcal{N}(0, 0.1^2)$
γ_s	Sum encoding strength; captures the overall query-key activation magnitude.	$\mathcal{N}(0, 0.1^2)$
α	Entanglement strength; scales the input-dependent rotation $RY(\alpha(q+k))$ between the two CNOTs, letting entanglement adapt to the input.	$\mathcal{N}(0, 0.1^2)$
β	Mixer angle; sets the $RX(2\beta)$ rotation applied to both qubits in the mixer layer.	$\mathcal{N}(0, 0.1^2)$

Algorithm 1 QPA per-dimension scoring function $\mu(q, k)$

Require: query scalar q , key scalar k , parameters $\theta_s, \gamma_d, \gamma_s, \alpha, \beta$
Ensure: score $\mu(q, k) = P(|00\rangle) + P(|11\rangle) \in [0, 1]$

- 1: $\lambda_1 \leftarrow \theta_s + \gamma_d + \gamma_s$ ▷ self-coefficient (Lemma 1, eq. 13)
 - 2: $\lambda_2 \leftarrow \gamma_s - \gamma_d$ ▷ cross-coefficient
 - 3: $\varphi_0 \leftarrow \frac{\pi}{4} + \lambda_1 q + \lambda_2 k$ ▷ encoding angle, qubit 0 (eq. 5), $\pi/4$ offset
 - 4: $\varphi_1 \leftarrow \frac{\pi}{4} + \lambda_2 q + \lambda_1 k$ ▷ encoding angle, qubit 1 (eq. 6), q, k swapped
 - 5: $|\psi\rangle \leftarrow RY(\varphi_0) \otimes RY(\varphi_1) |00\rangle$ ▷ product state after encoding
 - 6: $|\psi\rangle \leftarrow \text{CNOT}_{0 \rightarrow 1} |\psi\rangle$ ▷ entangling (eq. 7)
 - 7: $|\psi\rangle \leftarrow (I \otimes RY(\alpha(q+k))) |\psi\rangle$
 - 8: $|\psi\rangle \leftarrow \text{CNOT}_{1 \rightarrow 0} |\psi\rangle$
 - 9: $|\psi\rangle \leftarrow (RX(2\beta) \otimes RX(2\beta)) |\psi\rangle$ ▷ mixer (eq. 8)
 - 10: **return** $|\langle 00 | \psi \rangle|^2 + |\langle 11 | \psi \rangle|^2$ ▷ $\mu = P(|00\rangle) + P(|11\rangle)$ (eq. 10)
-

The QPA variant implemented from [29] underperformed the MedViT’s 10-epoch model from [4]. To improve the accuracy of the quantum model, we apply the suggestion from the ablation study and use a learnable temperature scalar τ to the QPA scoring function (S_d) before softmax, which is updated at each batch:

$$\tau = \frac{S_d}{\cos(q, k)} = S_d \left(\frac{q}{\|q\|_2} \cdot \frac{k^\top}{\|k\|_2} \right)^{-1}, \quad (\tau \leq 100).$$

We extended further the aggregation (summing) of the per-dimension measurement (see Equation 8) to get the attention score from 16 in [29] to MedViT’s [4] 32 channels,



to stop discarding half of the query-key pairs. The tuned variant shows an improvement over the base implementation, as displayed in Table 9.

$$A_{ij} = \sum_{d=1}^D \mu_d(Q_{i,d}, K_{j,d}), \quad D = \begin{cases} \min(16, d_h) = 16 & \text{(base variant)} \\ d_h & \text{(full-dimension aggregation)} \end{cases}$$

4.2 Quantum Softmax on GPU and Simulated QPU

The last quantum variant takes on implementing a custom quantum softmax function, inspired by the cornerstone work on Quantum Vision Transformer [28]. The quantum architectures evaluated in the work show comparable or superior performance on the MNIST datasets⁵, including RetinaMNIST.

The generalized attention score formula from [28, §1] is:

$$A_{ij} = \mathbf{x}_i^\top \mathbf{W} \mathbf{x}_j \quad (9)$$

$\mathbf{x}_i, \mathbf{x}_j$ being the patch vectors and $\mathbf{W}$ is a trainable orthogonal matrix. For the construction of $\mathbf{W}$, [28, §2] uses the Reconfigurable BeamSplitter (RBS) gate from Equation 10, which performs a rotation in the 2-dimensinoal plane of the statevectors $|01\rangle, |10\rangle$, fixing $|00\rangle$ and $|11\rangle$. We use `qml.SingleExcitation` from PennyLane⁶ to build the RBS gate.

$$\text{RBS}(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & \sin \theta & 0 \\ 0 & -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad (10)$$

The work explores several circuit layouts, built with the RBS gate: pyramid, butterfly, or X-shaped [28, §2.2]. Besides the 3 layouts from the cornerstone work, we use an additional brick-wall nearest-neighbor layout from [30, Figure 1], for the reason of computational efficiency, due to the fact that it only uses 2 RBS gates (see Table 5). The circuit blocks use 3-4 qubits and between 2 and 4 gates, as depicted in Table 5 and Figure 8.

⁵<https://keras.io/api/datasets/mnist/>

⁶<https://docs.pennylane.ai/en/stable/code/api/pennylane.SingleExcitation.html>

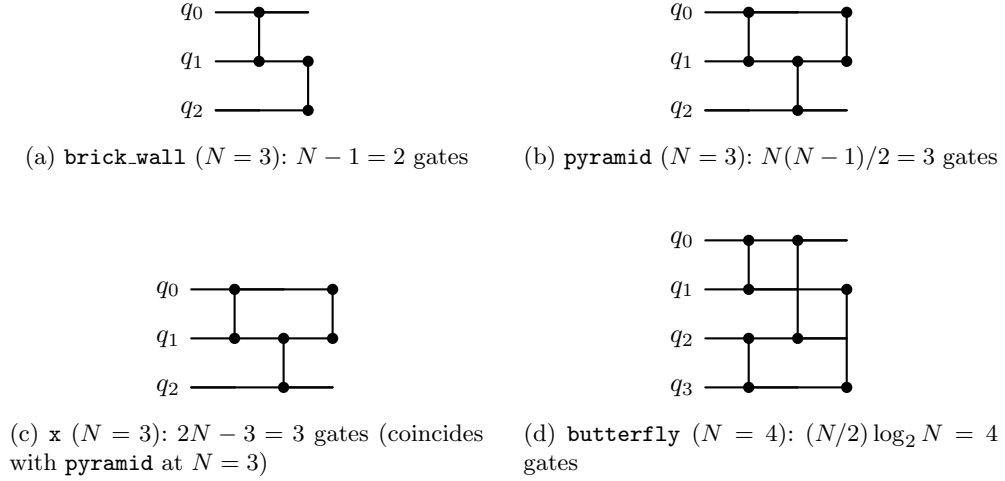


Fig. 8: The four RBS orthogonal-layer layouts at the qubit counts used by the quantum-softmax variant: **brick_wall**, **pyramid**, and **x** run at $N = 3$, while **butterfly** uses $N = 4$ qubits.

Table 5: Quantum-softmax RBS orthogonal-layer variants.

Layout	Qubits	Gates/block
brick_wall (default)	3	2
pyramid	3	3
butterfly	4	4
x	3	3

The quantum softmax variant from this section has 3 main setups:

- training and testing using GPU operations only; in PyTorch [16]
- training on GPU device, then testing using PennyLane's `default.qubit` simulator⁷
- performing both training and testing on simulated QPU

Instead of the canonical differentiable learning by backpropagation [31] (see Figure 9), which the GPU quantum softmax uses, the QPU simulation performs measurements via shot sampling [28, §3.2].

⁷<https://pennylane.ai/devices/default-qubit>

Table 6: Single-epoch operation count and time for QPU-sim at $S = 5000$ shots (RetinaMNIST, 1080 train, batch 10, input 224, $\tau_{\text{sim}} \approx 1.154$ ms per 3-qubit circuit at 5000 shots).

Step	Calculation	Result
Circuits / forward	$B \cdot H \cdot N$	11,760
Shot-meas / forward	$11,760 \times 5000$	58,800,000
Time / forward	$11,760 \times 1.154$ ms	13.57 s
Batches / epoch	1080/10	108
Circuits / epoch	$11,760 \times 108$	1,270,080
Shot-meas / epoch	$58,800,000 \times 108$	6,350,400,000
Time / epoch	$1,270,080 \times 1.154$ ms	≈ 1466 s ≈ 24.4 min

```

# GPU mode: EXACT statevector
# -> differentiable
@qml.qnode(dev, # no shots
            diff_method="backprop")
def circuit(amps, weights):
    qml.AmplitudeEmbedding(amps,
        wires)
    _apply_rbs_layout(weights,
        q_depth,
        pairs)
    return qml.state() # exact amps

# apply U(theta), Born rule in torch:
UT = circuit(basis, softmax_weights)
transformed = amps @ UT.real #
    smooth
probs = transformed ** 2 # Born
# grad flows -> softmax_weights
    updates

# QPU-sim: finite-shot SAMPLING
# NOT differentiable
@qml.qnode(dev, # dev has shots=N
            diff_method=None)
def circuit(amps, weights):
    qml.AmplitudeEmbedding(amps,
        wires)
    _apply_rbs_layout(weights,
        q_depth,
        pairs)
    return qml.probs(wires) #
        sampled

# gradient chain cut at the boundary:
amps = amps.detach()
with torch.no_grad():
    probs = circuit(amps,
        softmax_weights.detach())
# softmax_weights.grad is None ->
    frozen
    
```

(a) GPU mode: exact, differentiable.

(b) QPU-sim: sampled, non-differentiable.

Fig. 9: GPU mode doesn't use shots and reads the *exact* statevector. QPU-sim instead estimates probabilities from N finite *shots*.

Assuming a configuration with 1080 training images from RetinaMNIST, 10 batches, a 224 pixel input size, computed on a 3-qubit circuit, both the 512 and 5000 qpu-simulated epoch compute in a reasonable time (see Tables 6 and 8). The evaluation process equates to only 1 epoch on a 400-image test set. While the evaluation could be performed on a QPU simulator, training on QPU becomes unfeasible, as the shots measurements significantly increase the estimated time for an epoch, as shown in Table 8. In Section 5, we conducted our experiment on GPU training only, while we performed the evaluation step on both GPU and QPU simulation.

Table 7: Per-epoch training cost; each step needs $1 + 2N_{\text{params}} = 13$ circuit sets.

Step	Calculation	Result
Circuits / step	$N_{\text{circ}}(1 + 2N_{\text{params}}) = 11,760 \times 13$	152,880
Circuits / epoch	$152,880 \times 108$	16,511,040
Shot-meas / epoch ($S=512$)	$16,511,040 \times 512$	8.45×10^9
Shot-meas / epoch ($S=5000$)	$16,511,040 \times 5000$	8.26×10^{10}
Time / epoch ($S=512$)	$16,511,040 \times 0.275$ ms	≈ 4541 s ≈ 1.26 h
Time / epoch ($S=5000$)	$16,511,040 \times 1.154$ ms	$\approx 19,055$ s ≈ 5.29 h

Table 8: Per-epoch cost of QPU-sim training for the quantum-softmax variant (RetinaMNIST, 1080 train images, batch 10, input 224, $n_q = 3$).

Mode	Shots	Circuits/epoch	Shot-meas/epoch	Time/epoch
Evaluation	512	1.27 M	6.50×10^8	~ 5.8 min
Evaluation	5000	1.27 M	6.35×10^9	~ 24.4 min
Parameter-shift (trains)	512	16.5 M	8.45×10^9	~ 1.26 h
Parameter-shift (trains)	5000	16.5 M	8.26×10^{10}	~ 5.29 h

5 Results

Our report on the same 2 metrics as [4]. Accuracy (ACC) measures the overall fraction of correctly classified samples, but is sensitive to class imbalance [32]. The area under the ROC curve (AUC) [33, 34] is threshold-independent and robust to imbalance, quantifying the model’s ability to rank positives above negatives [35]; it is the standard discrimination metric in medical image classification.

The MedViT’s codebase implements 3 sizes for the training model, which differ by the number of training epochs. The small variant (MedViT-T) uses 10 epochs, the medium one (MedViT-S) uses 20, and the large version (MedViT-L) runs 30 epochs. From Table 9. We can see that MedViT’s methods outperformed the benchmark models on both the AUC and ACC metrics.

The experiment consists of the following variants, all derived from the MedViT small model MedViT-T in Table 9:

- **quantum_quanv_henderson2019** - the implementation using a quanvolutional layer from Section 3.3.
- **quantum_qpa_scoring** - integrates the existing attention code from the QPA [29], explained in Section 4.1.
- **quantum_qpa_scoring_tuned** - implements the dimension expansion and learnable temperature.
- **quantum_softmax_gpu_brick_wall** - uses the default brick wall pattern from [30]; both training and evaluation performed on GPU.
- **quantum_softmax_qpu_sim_brick_wall** - similar to quantum_softmax_gpu_brick_wall; evaluation performed on QPU simulator with 512 shots.
- **quantum_softmax_gpu_pyramid** - similar to quantum_softmax_gpu_brick_wall, but it uses the pyramid circuit block from Figure 8b.



- **quantum_softmax_gpu_x** - similar to `quantum_softmax_gpu_brick_wall`, but it uses the x circuit block from Figure 8c.
- **quantum_softmax_gpu_butterfly** - similar to `quantum_softmax_gpu_brick_wall`, but it uses the butterfly circuit block from Figure 8d.

Overall, the quantum variants slightly underperformed the classical models. Among the reasons for the results, we enumerate the untrainability of the quanvolutional layer from Section 3.1; the low-contrast bounded scoring of the base QPA variant (Section 4.1), which produces near-uniform attention and underfits; the limited expressivity and few trainable parameters of the quantum blocks; the small circuit width (3–4 qubits) imposed by simulation cost.

The results from Table 9 show an improved accuracy for the `quantum_softmax_qpu_sim_brick_wall` and `quantum_softmax_gpu_pyramid` variants, but the AUC metric shows weaker discrimination between classes, which could be due to the dataset’s class imbalance [33]; Table 10 shows that the majority (45%) of the images belong to 1 class only.

Table 9: Performance comparison on RetinaMNIST. In parentheses, we have the input size for the given method.

Methods	AUC	ACC
ResNet-18 (28) [6]	0.717	0.524
ResNet-18 (224) [6]	0.710	0.493
ResNet-50 (28) [6]	0.726	0.528
ResNet-50 (224) [6]	0.716	0.511
auto-sklearn [6]	0.690	0.515
AutoKeras [6]	0.719	0.503
Google AutoML [6]	0.750	0.531
MedViT-T (224) [6]	0.752	0.534
MedViT-S (224) [6]	0.773	0.561
MedViT-L (224) [6]	0.754	0.552
quantum_quanv_henderson2019	0.623 ↓	0.443 ↓
quantum_qpa_scoring	0.713 ↓	0.438 ↓
quantum_qpa_scoring_tuned	0.693	0.507
quantum_softmax_gpu_brick_wall	0.731 ↓	0.540 ↑
quantum_softmax_qpu_sim_brick_wall	0.729	0.517
quantum_softmax_gpu_pyramid	0.709 ↓	0.542 ↑
quantum_softmax_gpu_butterfly	0.697	0.497
quantum_softmax_gpu_x	0.687	0.532

Table 10: Class distribution of RetinaMNIST [6] (5 ordinal grades of diabetic retinopathy). The training set is markedly imbalanced: the majority grade 0 (45.0%) outnumbers the minority grade 4 (6.1%) by $\approx 7.4\times$.

Class (grade)	Train	Val	Test	Total	Train %
0	486	54	174	714	45.0%
1	128	12	46	186	11.9%
2	206	28	92	326	19.1%
3	194	20	68	282	18.0%
4	66	6	20	92	6.1%
Total	1080	120	400	1600	100%

6 Future Work

To further explore the potential quantum advantage for a quantum MedViT, the future contributions will transform other blocks from the architecture into quantum or quantum-like functions, including the Multi-Head Convolutional Attention (MHCA) [23, 36], the feed-forward (MLP) layers [37, 38], the patch-embedding convolutions [20], the value projection [39], and the classification head [36, 40]. Then, our contribution requires evaluation against more benchmarks (both quantum and classical architectures) and datasets (from MedMNISTV2 [6] or custom medical datasets).

7 Conclusion

In this report, we translate the self-attention mechanism and initial convolutional block for image encoding into quantum. We use the quanvolutional layer [20], parametrised self-attention [29], and a quantum orthogonal-layer softmax [28] that replaces the classical softmax with the Born rule probabilities of an amplitude-encoded score evolved through a trainable RBS circuit. In our training and evaluation, we use both QPU and QPU simulated measurements for the quantum blocks. Our contribution shows comparable, but inferior results to the current state-of-the-art MedViT. Further development of quantum blocks from this MedViT architecture could showcase an improvement in performance on relevant metrics.

References

- [1] Zaman, K., Marchisio, A., Hanif, M.A., Shafique, M.: A Survey on Quantum Machine Learning: Current Trends, Challenges, Opportunities, and the Road Ahead. Preprint at <https://arxiv.org/abs/2310.10315> (2025). <https://doi.org/10.48550/arXiv.2310.10315>
- [2] Gokhale, S.S., Dhote, R.R., Delhibabu, R.: A review of quantum machine learning algorithms, applications, and emerging advantages. *Discovery Computing* **29**, 226 (2026) <https://doi.org/10.1007/s10791-026-10085-1>



- [3] Thudumu, S., Fisher, J., Du, H.: Supervised Quantum Machine Learning: A Future Outlook from Qubits to Enterprise Applications. Preprint at <https://arxiv.org/abs/2505.24765> (2025)
- [4] Manzari, O.N., Ahmadabadi, H., Kashiani, H., Shokouhi, S.B., Ayatollahi, A.: MedViT: A robust vision transformer for generalized medical image classification. *Computers in Biology and Medicine* **157**, 106791 (2023) <https://doi.org/10.1016/j.compbiomed.2023.106791>
- [5] Manzari, O.N., Asgariandehkordi, H., Koleilat, T., Xiao, Y., Rivaz, H.: Medical image classification with KAN-integrated transformers and dilated neighborhood attention. *Applied Soft Computing* (2025) <https://doi.org/10.1016/j.asoc.2025.114045>
- [6] Yang, J., Shi, R., Wei, D., *et al.*: MedMNISTv2 – a large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data* **10**, 41 (2023) <https://doi.org/10.1038/s41597-022-01721-8>
- [7] Kadric, N., Akagic, A., Kapo, M.: Improving Diagnostic Accuracy of Pigmented Skin Lesions With CNNs: an Application on the DermaMNIST Dataset. Preprint at <https://arxiv.org/abs/2507.12961> (2025). <https://doi.org/10.48550/arXiv.2507.12961>
- [8] Caetano, F., Abdi, L., Viviers, C., Valiuddin, A., Sommen, F.: MedSymmFlow: Bridging generative modeling and classification in medical imaging through symmetrical flow matching. In: *Proc. DGM4MICCAI 2025* (2025). https://doi.org/10.1007/978-3-032-05472-2_4
- [9] LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proceedings of the IEEE* **86**(11), 2278–2324 (1998) <https://doi.org/10.1109/5.726791>
- [10] Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift. In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, vol. 37, pp. 448–456. PMLR, ??? (2015)
- [11] PyTorch Contributors: torch.nn.Conv2d — PyTorch 2.12 documentation. <https://docs.pytorch.org/docs/2.12/generated/torch.nn.Conv2d.html>. Accessed: 2026-06-01 (2025)
- [12] PyTorch Contributors: torch.nn.BatchNorm2d — PyTorch 2.12 documentation. <https://docs.pytorch.org/docs/2.12/generated/torch.nn.BatchNorm2d.html>. Accessed: 2026-06-01 (2025)
- [13] PyTorch Contributors: torch.nn.ReLU — PyTorch 2.12 documentation. <https://>

- docs.pytorch.org/docs/2.12/generated/torch.nn.ReLU.html. Accessed: 2026-06-01 (2025)
- [14] Nair, V., Hinton, G.E.: Rectified linear units improve restricted boltzmann machines. In: Proceedings of the 27th International Conference on Machine Learning (ICML), pp. 807–814 (2010)
 - [15] Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information: 10th Anniversary Edition. Cambridge University Press, ??? (2010)
 - [16] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S.: Pytorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems 32 (NeurIPS), pp. 8024–8035 (2019)
 - [17] Glorot, X., Bordes, A., Bengio, Y.: Deep sparse rectifier neural networks. In: Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), pp. 315–323 (2011)
 - [18] Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press, ??? (2016). <http://www.deeplearningbook.org>
 - [19] Rosen, K.H.: Discrete Mathematics and Its Applications, 8th edn. McGraw-Hill Education, ??? (2018)
 - [20] Henderson, M., Shakya, S., Pradhan, S., Cook, T.: Quadvolutional neural networks: Powering image recognition with quantum circuits. Preprint at <https://arxiv.org/abs/1904.04767> (2019). <https://doi.org/10.48550/arXiv.1904.04767>
 - [21] Erdos, P., Renyi, A.: On the evolution of random graphs. Publications of the Mathematical Institute of the Hungarian Academy of Sciences **5**(1), 17–60 (1960)
 - [22] Bergholm, V., Izaac, J., Schuld, M., Gogolin, C., Ahmed, S., Ajith, V., Alam, M.S., Alonso-Linaje, G., AkashNarayanan, B., Asadi, A., Arrazola, J.M., Azad, U., Banning, S., Blank, C., Bromley, T.R., Cordier, B.A., Ceroni, J., Delgado, A., Di Matteo, O., Dusko, A., Garg, T., Guala, D., Hayes, A., Hill, R., Ijaz, A., Isacsson, T., Ittah, D., Jahangiri, S., Jain, P., Jiang, E., Khandelwal, A., Kottmann, K., Lang, R.A., Lee, C., Loke, T., Lowe, A., McKiernan, K., Meyer, J.J., Montañez-Barrera, J.A., Moyard, R., Niu, Z., O’Riordan, L.J., Oud, S., Panigrahi, A., Park, C.-Y., Polatajko, D., Quesada, N., Roberts, C., Sá, N., Schoch, I., Shi, B., Shu, S., Sim, S., Singh, A., Strandberg, I., Soni, J., Száva, A., Thabet, S., Vargas-Hernández, R.A., Vincent, T., Vitucci, N., Weber, M., Wierichs, D., Wiersema, R., Willmann, M., Wong, V., Zhang, S., Killoran, N.: PennyLane: Automatic differentiation of hybrid quantum-classical computations. arXiv preprint arXiv:1811.04968 (2018) [arXiv:1811.04968](https://arxiv.org/abs/1811.04968) [quant-ph]

- [23] Schuld, M., Killoran, N.: Quantum machine learning in feature hilbert spaces. *Physical Review Letters* **122**(4), 040504 (2019) <https://doi.org/10.1103/PhysRevLett.122.040504>
- [24] Havlíček, V., Córcoles, A.D., Temme, K., Harrow, A.W., Kandala, A., Chow, J.M., Gambetta, J.M.: Supervised learning with quantum-enhanced feature spaces. *Nature* **567**(7747), 209–212 (2019) <https://doi.org/10.1038/s41586-019-0980-2>
- [25] Weigold, M., Barzen, J., Leymann, F., Salm, M.: Encoding patterns for quantum algorithms. *IET Quantum Communication* **2**(4), 141–152 (2021) <https://doi.org/10.1049/qtc2.12032>
- [26] Hur, T., Kim, L., Park, D.K.: Quantum convolutional neural network for classical data classification. *Quantum Machine Intelligence* **4**(1), 3 (2022) <https://doi.org/10.1007/s42484-021-00061-x>
- [27] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: *Advances in Neural Information Processing Systems*, vol. 30 (2017)
- [28] Cherrat, E.A., Kerenidis, I., Mathur, N., Landman, J., Strahm, M., Li, Y.Y.: Quantum vision transformers. *Quantum* **8**, 1265 (2024) <https://doi.org/10.48550/arXiv.2209.08167>
- [29] Zhang, W., Wang, J., Ye, T., Liao, C.: Quantum Parameterized Self-Attention Network for Image Classification (2026). <https://doi.org/10.48550/arXiv.2605.25365>
- [30] Kandala, A., Mezzacapo, A., Temme, K., Takita, M., Brink, M., Chow, J.M., Gambetta, J.M.: Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *Nature* **549**(7671), 242–246 (2017)
- [31] Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. *Nature* **323**(6088), 533–536 (1986) <https://doi.org/10.1038/323533a0>
- [32] Sokolova, M., Lapalme, G.: A systematic analysis of performance measures for classification tasks. *Information Processing & Management* **45**(4), 427–437 (2009) <https://doi.org/10.1016/j.ipm.2009.03.002>
- [33] Fawcett, T.: An introduction to ROC analysis. *Pattern Recognition Letters* **27**(8), 861–874 (2006) <https://doi.org/10.1016/j.patrec.2005.10.010>
- [34] Hanley, J.A., McNeil, B.J.: The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology* **143**(1), 29–36 (1982) <https://doi.org/10.1148/radiology.143.1.7063747>

- [35] Bradley, A.P.: The use of the area under the ROC curve in the evaluation of machine learning algorithms. *Pattern Recognition* **30**(7), 1145–1159 (1997) [https://doi.org/10.1016/S0031-3203\(96\)00142-2](https://doi.org/10.1016/S0031-3203(96)00142-2)
- [36] Schuld, M., Bocharov, A., Svore, K.M., Wiebe, N.: Circuit-centric quantum classifiers. *Physical Review A* **101**(3), 032308 (2020) <https://doi.org/10.1103/PhysRevA.101.032308>
- [37] Mari, A., Bromley, T.R., Izaac, J., Schuld, M., Killoran, N.: Transfer learning in hybrid classical-quantum neural networks. *Quantum* **4**, 340 (2020) <https://doi.org/10.22331/q-2020-10-09-340>
- [38] Pérez-Salinas, A., Cervera-Lierta, A., Gil-Fuster, E., Latorre, J.I.: Data re-uploading for a universal quantum classifier. *Quantum* **4**, 226 (2020) <https://doi.org/10.22331/q-2020-02-06-226>
- [39] Kerenidis, I., Landman, J., Mathur, N.: Classical and quantum algorithms for orthogonal neural networks. *arXiv preprint arXiv:2106.07198* (2021)
- [40] Farhi, E., Neven, H.: Classification with quantum neural networks on near term processors. *arXiv preprint arXiv:1802.06002* (2018)